

Note: I'll redo all the randomness and payout scheme logic because the current status of the project isn't satisfying the assignment requirements.

My approach to randomness:

All the cells, "Slots", in this project have their own "SlotValue" that's taking a completely random integer number between 0 and 3.

Code example from *SlotState.cs*:

```
slotValue = Random.Range(0, maxSlotValue);
```

Description of payout schemes logics

This was the hardest part to plan and figure out for me because I'm unsure how to mathematically approach this. I ended up AI-generating this logic with GitHub Copilot just to get the requirement in.

Pattern	Default Multiplier	Inspector Variable
Row Match	5x	rowMatchMultiplier
Column Match	5x	columnMatchMultiplier
Diagonal Match	10x	diagonalMatchMultiplier
Cluster	2x per symbol	clusterMultiplier
Four Corners	15x	cornerMatchMultiplier
X Pattern	20x	xPatternMultiplier
Plus Pattern	12x	plusPatternMultiplier
Frame Pattern	18x	framePatternMultiplier

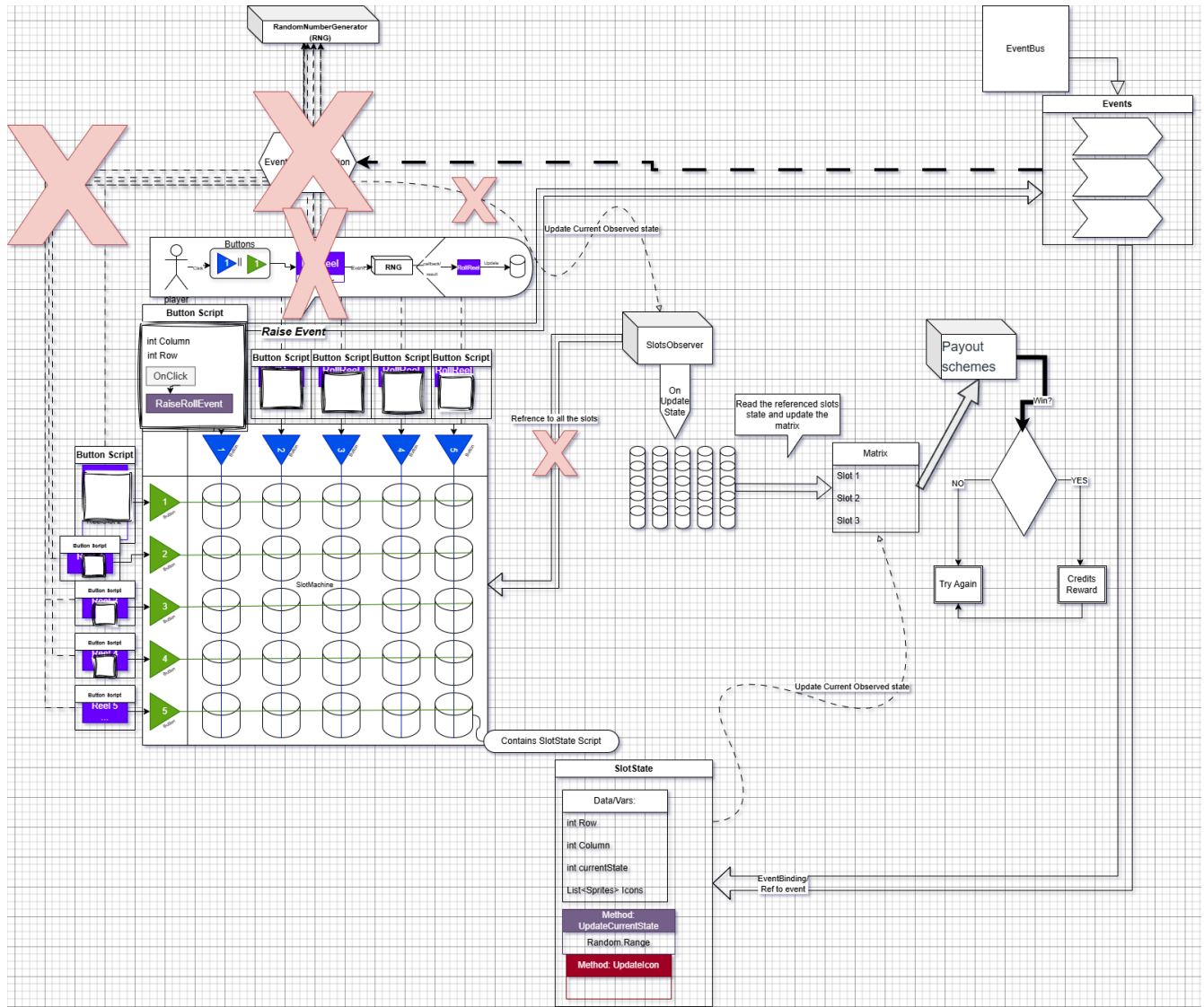
How you ensured "profitability":

Currently, profitability is ensured for the “casino” in the way that player will, with high probability, always lose (which isn’t what my intended design goal was supposed to be, but implementation time led to it becoming like this).

Challenges:

I spent a majority of my time mapping out how each part of the system should be constructed, because my goal was to have each class deal with “separation of concerns” and mostly communicate through events.

Hence I brainstormed/mapped out the layout of the system first in draw.io.



The “X” represents logic I decided to redo, link to diagram

<https://drive.google.com/file/d/1uECeQVjEK1EYnd8GqLquwcgBzXoQTyyW/view?usp=sharing>

The next challenge after implementing was the randomness in the slot machine. This is because of these design requirements:

“Each scheme should have its own **reward logic**, but the overall system must be designed so that **the machine always makes money over time**. This involves:

- Designing **probability distributions** for symbol appearances.

- Implementing **expected value calculations** for each scheme.
- Ensuring **player retention** through perceived fairness and excitement.”

I want to go for the same implementation as shown in one of our last lectures: having the controlled random ranges inside conditional statements, so it's easier to delegate each probable outcome – but I didn't manage to implement this before the assignment deadline.

I think “Ensuring **player retention**” is the hardest one because I'm still unsure how to properly do the payout scheme logic. The current implementation isn't functional in the project.